IPEST- SUPPORT DE REVISION Python

réalisé par S.Darragi





1 Les types élémentaires sous Python

- Les entiers : classe <int> Exemple x=3
- Les réels : classe <float> Exemple x=2.3
- Les booléens : classe <bool> Exemple x=True , y=False
- Les complexe: classe <complex> Exemple Z=2+5*1j (2+5i) , l'imaginaire pur i est désigné par 1j
- Les chaînes de caractères : classe <str> Exemple ch='bonjour'

Toutefois, il est possible de changer de type, Attention les objets converti sont de nouveaux objets!

```
>>>x=2
>>>x
2
>>>float(x)
2.0
>>>str(x)
'2.0'
>>>x
```

2 !!!! x n'a pas changer de type

Attention quelques changement de type ne sont pas autorisé !

Exemple

```
>>>Z=1+3*1j
>>>int(Z)
Traceback (most recent call last):
File "<pyshell#7>", line 1, in <module> int(z)
TypeError: can't convert complex to int
```

Mais ...

```
>>>str(Z)
'(1+4j)'
```

1.1 Les opérateurs prédéfinis +, -, * , **, //, / ,%...

Soient x et y deux objets numériques

- Addition : x+y
- Soustraction : x-y

- Produit : x*y
- Puissance : x** y ou encore pow(x,y)
- Division entière : Quotient : x//y
- Reste de diviiosn entière : x%y
- divmod(x,y) retourne un couple (quotient , reste)

>>> divmod(13,4)
(3, 1)
Notation scientifique

>>> 2 e5 2 exposant 5 cad 200000.0)

>>>10e-5 (10 exposant -5)
D'autres fonctions seront disponibles dans la bibliothèque math
>>> from math import *
>>> sqrt(5)
2.23606797749979
>>> cos(pi/2)
6.123233995736766e-17# le zero en numerique n'est pas un vrai zero !!
>>> sin(pi)
1.2246467991473532e-16
>>> cos(pi)
-1.0
>>> sin(pi)
1.2246467991473532e-16

1.2 Les opérateurs logiques et de comparaison

1.2.1 Les opérateurs logiques et , ou , et non

Les opérateurs logique s'appliquant à deux objets A et B sont
L'operateur ET: A et B –> A and B
L'operateur OU: A ou B –> A or B
L'operateur NON: non (A)–> not(A)

1.2.2 Les opérateurs de comparison logiques <, <=, >, >=, == et !=

Les opérateurs de comparaison entre deux objets A et B sont respectivement :
Test d'égalité : A=B–> A==B
Test de différence :A<>B–> A!=B
Strcitement inferieur A<B–> A<B
Inferieur ou égal: A<=B–> A<= b
Strcitement superieur: A>B–> A >B
Superieur ou égal : A>=B–> A >=B

2 Syntaxe Python

Nous rappelons dans cette section la syntaxe de base du langage python

2.1 L'opération élémentaire d'affectation

L'opération élementaire d'affectation est donné par l'opérateur "="

```
>>>x=5
x
>>>y=x
>>>x,y
(5,5)
>>>x=y=z=t=0
>>>x,y,z,t
(0,0,0,0)
>>>x,y,z,t=2,4,6,1
>>>x,y,z,t
(2,4,6,1)
>>>x,y=5,8
>>>x,y=y,x
>>>x,y
(8,5)
```

Extension pour les opérations élémentaires +,-,*...

```
>>>X=2,Y=3
>>>X+=1 #X=X+1
>>>X
3
>>>Y*=2 #Y=Y*2
>>>Y
6
>>>Z=X+Y
>>>Z
9
>>>Z-=1 #Z=Z-1
>>>Z
8
>>>Z//=2 #Z=Z//2
>>>Z
4
>>>Z%=3 #Z=Z%3
>>>Z
1
```

2.2 Les opérations d'entrée/ sortie

Print & Input

```
>>>print("ceci est un texte")
ceci est un texte
>>>print("ceci est \n un texte avec retour à la ligne")
ceci est
un texte avec retour à la ligne
```

L'intruction *print* permet d'afficher le texte entre double quotes à l'écran

```
>>>ch=input("saisir votre texte")
```

L'instruction *input* donne la main à l'utilisateur pour introduire ses données, les valeurs introduites sont concidérées par défaut des chaines de caractères, toutefois nous pouvons changer ces chaines en entier ou en réel avec *int* et *float*

2.3 Les structures conditionnelles

Une structure conditionnelle sous Python s'exprime avec la clause *if cond : Instruction else : Instruction 2*, toutefois il faut faire très attention à l'indentation elle s'écrira sous la forme suivante:

2.3.1 Structure Conditionnelle Simple

if *cond* :
    Instruction 1
else :
    Instruction 2

2.3.2 Structure Conditionnelle Imbriquée

if *cond_ 1* :
    Instruction 1
elif *cond_ 2*:
    Instruction 2
elif *cond_ 3*:
    Instruction 3
    ...
else :
    Instruction n

2.4 Les structures itératives Boucle Pour... et Tant que

2.4.1 La boucle Pour ...

for i in range(a,b,c):
    Instruction 1
    Instruction 2
    Instruction 2

Nous notons par range(a,b,c) , l'intervalle [a,b[, a inclu, b exclu avec un pas égal à c, et voici des exemples

- range(5) designera les entiers 0,1,2,3,4 , *la borne a prendra par défaut 0*
- range(1, 10) désignera les entiers 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- range(1, 11, 2) désignera les entiers 1, 3, 5, 7, 9
- range(10, 0, -1) désignera les entiers 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

2.4.2 La boucle Tant que ...

while *cond_ vrai*:
    Instruction 1
    Instruction 2
    ...
    Instruction n

2.5 Les sous-programmes

Pour définir une fonction sous Python, nous pouvons le faire de deux manières en utilisant le mot clé *def* ou *lambda.*

def *(param_ 1,param_ 2n...,param_ n)*:
    Instruction 1
    Instruction 2
    ...
    return(Résulat(s))

Exemple

```
def factoriel (n):
    f=1
    for i in range(1,n+1):\\
        f * =i
    return(f)
>>>factoriel(0)
1
>>>factoriel(5)
120
```

Ou encore définir une instruction avec Lambda , c'est utile surtout dans le cas de fonction mathématiques !

lambda*(param_ 1,param_ 2...,param_ n):*
Instruction(s) élémentaire(s)

Exemple

```
F= lambda x:x**2
G=lambda x,y= x**2+y**3
>>>F(4)
16
>>>G(2,3)
35
```

3 Les conteneurs Python

Un conteneur *C* sous python est un itérable de longueur *len(C)* qui commence par l'indice 0 et se termine par l'indice *len(C)-1* . D'une façon générale la lecture des éléments d'un conteneur se fait de la même manière quelque soit le conteneur (chaîne de caracères, tuple, liste..):

Extraction des éléments d'un Conteneur C contenant n objets

- C[0]: extraction du premier élément de C
- C[2:5:2] : sélection du 3eme au 6eme (exclu) éléments par pas de 2.
- C[-1] : dernier élément
- C[-2] : avant dernier élément
- C[1:]: sélection du deuxième élément jusqu'à la fin de C
- C[:3] : sélection du début jusqu'au troisième élément

3.1 Les chaines de caracères

Une chaîne de caractère est un conteneur "itérable" non modifiable. Soit *s* une chaine de caractère

- «s.upper()» Mettre tout les caractères alphabetiques de s en majuscule
- «s.lower() » Mettre tout en minuscule
- «s.title()» Mettre la première lettre de chaque mots en majuscule, le reste en minuscule
- «s.swapcase()» Mettre les minuscules en majuscule et les majuscules en minuscule
- «s.islower()» Indique si toute la chaine est en minuscule (True/False)
- «s.isupper()» Indique si tt est en majuscule (True/False)

- «s.isalpha()» Indique si la chaine contient uniquement des caractères alphabétique (True/False)
- «s.isdigit()» Indique si la chaine contient uniquement des caractères numériques (True/False)
- «s.isalnum()» Indique si la chaine contient uniquement des caractères alphanumérique (True/False)
- «s.split(';')» Découper une chaine en liste en fonction du séparateur ';' split() tout court découpe selon le caractère espace
- «s.strip()» : si s contient une caractère de saut de ligne '\n', il sera supprimé!

3.2 Les tuples

Un tuple est un conteneur non modifiable, un ensemble d'objet entre ( ).

- T = (el1 ,el2 , el3) définit un tuple T.
- «tuple(L)» transforme la séquence L en un tuple.
- «Permuter des données» var1, var2 = var2, var1
- T=2, désigne un tuple contenant un seul élément attention à la vrigule à la fin

3.3 Les Listes

Une liste est un conteneur modifiable, c'est une séquence d'éléments entre [ ].

- «T = [el1 ,el2 , el3]» créer une liste L
- «list(T)» Transforme un tuple en une liste, ou une chaine en une liste
- «list(range(10))» : liste de 0 à 9
- «Concaténation» L1+L2

Extraction des éléments d'une liste Soit L=[2,4,-6,-4,3,2,1,6,-11]

- L[0] : extraction du premier élément dans L: 2
- L[2:5:2] : sélection du 3eme au 6eme (exclu) éléments par pas de 2 .
- L[-1] : dernier élément -11
- L[-2] : avant dernier élément
- L[1:] sélection du deuxième élément jusqu'à la fin de L.
- L[:4]: sélection du début jusqu'au quatrième élément
- L[:] : retourne une liste contenant tous les éléments de L, une copie profonde de L

3.4 Les dictionnaires

Un dictionnaire D est un ensemble de couples 'clé:valeur'

- «D = {"clé1": valeur1, "clé2": valeur2, "clé3": valeur3}» Définit un dictionnaire D.
- « print(D["cle1"])» Affiche valeur1
- «D["cle"] = valeur» Ajoute un élément ou modifie la valeur de la cle en question
- «D.keys()» renvoie une liste contenant les clés.
- «D.values()» renvoi une liste contenant les valeurs du dictionnaire.
- «D.has_key(clé)» revoit le booléen True si le dictionnaire contient la clé ou la valeur, False sinon.

- «D.get(clé, valsinon)» renvoit la valeur d'une clé ou valsinon si la clé n'existe pas.
- «len(D)» renvoi la taille du dico (nombre de clés)
- « D.items()» renvoi une liste de tuple, chaque tuple contenant clé puis valeur
- « del (D[clé])» supprime un élément du dictionnaire (clé et valeur), s'il existe sinon une erreur est déclenchée

4 Les fichiers

Un fichier texte est également un conteneur itérable !, nous pourvons créer un fichier, l'ouvrir s'il existe , le fermer aprés ouverture , le lire , écrire ...;

nom_logique = open("nom_physique", "mode d'ouverture")» pour ouvrir un fichier en lecture Les modes d'ouverture sont: r lecture seule, w écriture, a pour ajouter du texte, s'il n'existe pas il sera créer (mode w et a)

contenu = nom_logique .read() pour la lecture du fichier.

une_ligne = nom_logique .readline() pour lire une seule ligne.

lignes = nom_logique .readlines()

nom_logique.close() pour fermer le fichier.

nom_logique .write("Ceci est un texte") pour écrire du texte.